KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

파이썬 기초
변수, 타입, 연산

값에 이름 붙이고, 타입 알고,
계산하고, 참 거짓 따지기

```
age = 25          → 넣기
type(age)         → int
7 // 3            → 2
age >= 20         → True
```

오늘의 순서

1

변수에 값 넣기

= 로 넣기, 값 보기, 다시 넣기, NameError

4–9쪽

2

변수 타입

int, float, str, bool, type(), 타입 바꾸기

10–14쪽

3

사칙연산

+ - * /, 몫과 나머지, 계산 순서, +=, 글자 연산

15–21쪽

4

비교와 논리연산

== 와 =, True와 False, and, or, not

22–30쪽

3쪽은 노트북 준비, 31–32쪽은 막힐 때와 한 장 요약이에요.

들어가기

10에서 이어서 해요

- **hello-uv 노트북, 커널 .venv** — 09 · 25–27쪽
- **변수 이름 규칙과 표기법** — 10 · 19–21쪽
- **자료형과 type()** — 10 · 22–23쪽

기억이 안 나면 그 쪽을 먼저 다시 봐요.

새 노트북 만들기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 새 파일 `basics.ipynb`
3. 오른쪽 위 커널로 .venv 고르기
4. 셀에 적고 `Shift` + `Enter` 로 실행

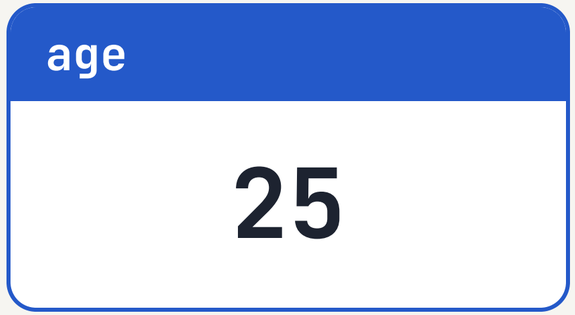
이번 챕터의 코드는 모두 노트북 셀에 적어요.

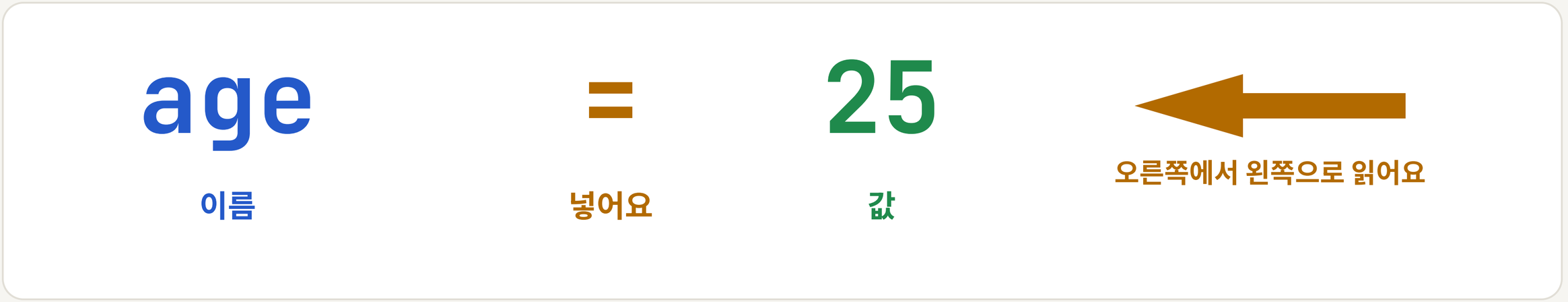
1. 변수에 값 넣기

= 는 오른쪽 값을 왼쪽 이름에 넣어요

```
[1]  age = 25                                                    Python
```

넣기만 해서 출력은 없어요. 값은 커널이 기억해요.





수학의 '같다'가 아니에요. 25를 age라는 이름에 넣어요.

1. 변수에 값 넣기

넣은 값은 두 가지 방법으로 봐요

셀 마지막 줄에 이름만 적기

[2]
```python
age
```

```
25
```

print()로 꺼내 보기

[3]
```python
print(age)
print("나이:", age)
```

```
25
나이: 25
```

이름만 적기

10에서 본 대로 셀의 마지막 줄이
값이면 print 없이도 보여 줘요.

print("설명", 값)

쉼표로 나누면 한 칸 띄워서 이어 보여 줘요.
따옴표 안 글자는 그대로, age는 값 25로 바뀌어요.

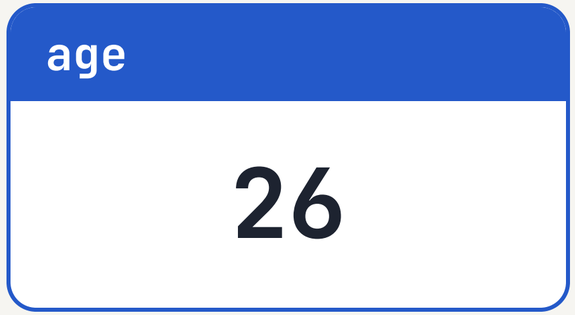
빨리 확인할 땐 이름만, 설명을 붙일 땐 print("설명", 값)을 써요.

1. 변수에 값 넣기

다시 넣으면 새 값으로 바뀌어요

[4]
```python
age = 26
age
```

```
26
```



예전 값 ~~25~~

상자 하나에는 값이 하나만 들어가요.
새 값을 넣으면 예전 값 25는 사라져요.

같은 이름에 넣을수록 마지막에 넣은 값만 남아요.

1. 변수에 값 넣기

age = age + 1은 오른쪽부터 계산해요

[5]
```
age = age + 1
age
```
Python

```
27
```

age

27

1 오른쪽 age를 읽어요

`age → 26`

2 계산해요

`26 + 1 → 27`

3 왼쪽 age에 넣어요

`age ← 27`

10에서 실행한 x = x + 1도 같은 순서로 계산했어요.

1. 변수에 값 넣기

이름만 적으면 마지막 줄만 보여요

age는 안 보여요

[6]
```python
user_name = "kim"
age
user_name
```

```
'kim'
```

print는 둘 다 보여요

[7]
```python
print(age)
print(user_name)
```

```
27
kim
```

가운데 줄 age는 실행은 되지만 화면에 나오지 않아요.

`'kim'`	이름만 적기는 따옴표까지
`kim`	print는 글자만

여러 값을 보고 싶으면 줄마다 print()를 써요.

1. 변수에 값 넣기

이름을 틀리게 적으면 NameError가 나요

```
[8]  Age                                                    Python
```

```
NameError: name 'Age' is not defined
```

대문자와 소문자는 다른 이름이에요

`age` 넣은 이름

`Age` 한 번도 넣은 적 없는 이름

이럴 때도 같은 오류가 나요

- 철자를 틀렸어요 (agee)
- 넣는 셀을 아직 실행하지 않았어요
- Restart 뒤에 아래 셀만 실행했어요

NameError가 나면 철자부터 보고, 넣는 셀을 실행했는지 봐요.

2. 변수 타입

값마다 타입이 있어요

[1]
```python
age = 25
height = 172.5
user_name = "kim"
is_member = False
print(type(age))
print(type(height))
print(type(user_name))
print(type(is_member))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

정수 int

`25`

소수점 없는 숫자

실수 float

`172.5`

소수점 있는 숫자

문자열 str

`"kim"`

따옴표로 감싼 글자

불리언 bool

`False`

True 또는 False

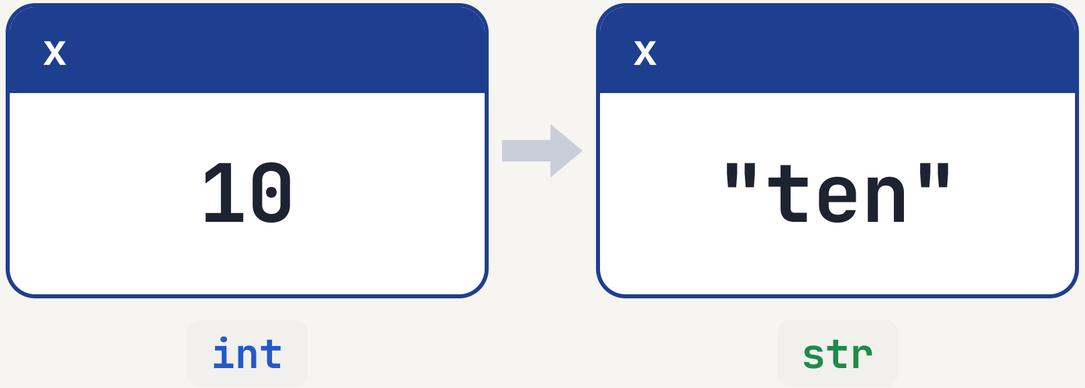
타입은 값의 종류예요. 10에서 본 자료형과 같은 말이에요.

2. 변수 타입

타입은 이름이 아니라 값을 따라가요

[2]
```python
x = 10
print(type(x))
x = "ten"
print(type(x))
```

```
<class 'int'>
<class 'str'>
```



이름 x는 그대로인데, 넣은 값이 바뀌니 타입도 바뀌었어요.
타입을 미리 정하지 않아도 넣는 값을 보고 정해져요.

한 이름에는 한 가지 뜻의 값만 넣어요. 헷갈리면 type()으로 확인해요.

2. 변수 타입

따옴표 하나로 숫자가 글자가 돼요

[3]
```python
print(type(25))
print(type("25"))
print(25 + 1)
print("25" + "1")
```

```
<class 'int'>
<class 'str'>
26
251
```

25 int

25 + 1 → **26**

숫자끼리 더해요

"25" str

"25" + "1" → **251**

글자끼리 이어 붙여요

겉모습이 숫자여도 따옴표 안에 있으면 str이에요.

2. 변수 타입

int(), float(), str()로 타입을 바꿔요

[4]
```python
print(int("3") + 3)
print(str(3) + "3")
print(float("3.5"))
print(int(3.9))
```

```
6
33
3.5
3
```

글자를 정수로

`int("3")` → `3`

숫자를 글자로

`str(3)` → `'3'`

글자를 실수로

`float("3.5")` → `3.5`

소수점 아래는 버려요

`int(3.9)` → `3`

바꾸고 싶은 타입 이름을 함수처럼 쓰고 괄호 안에 값을 넣어요.

2. 변수 타입

숫자로 못 읽는 글자는 ValueError가 나요

```
[5] int("삼")                                                    Python
```

```
ValueError: invalid literal for int() with base 10: '삼'
```

```
[6] int("3.5")                                                   Python
```

```
ValueError: invalid literal for int() with base 10: '3.5'
```

한글 숫자는 못 읽어요

int()는 0부터 9까지 숫자 글자만 읽어요.
"삼"은 사람에게만 숫자예요.

소수점 글자는 float()로

`float("3.5")` → **3.5**

앞 장에서 실행한 결과예요

ValueError는 타입은 맞는데 값을 바꿀 수 없다는 뜻이에요.

3. 사칙연산

더하기, 빼기, 곱하기, 나누기

[1]
```python
print(7 + 3)
print(7 - 3)
print(7 * 3)
print(7 / 3)
```

```
10
4
21
2.3333333333333335
```

+ 더하기

`7 + 3` → `10`

- 빼기

`7 - 3` → `4`

***** 곱하기

× 대신 별표

`7 * 3` → `21`

/ 나누기

÷ 대신 빗금

`7 / 3` → `2.333…`

키보드에 ×, ÷가 없어서 곱하기는 *, 나누기는 / 를 써요.

3. 사칙연산

/ 로 나누면 항상 소수점이 붙어요

```
[2]  6 / 2                                    Python

     3.0
```

딱 나눠떨어져도

`6 / 2` → `3.0`

3이 아니라 3.0이에요.

3 정수 `int`

소수점 없는 숫자

3.0 실수 `float`

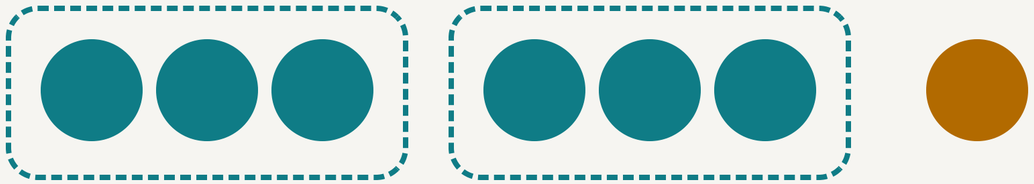
소수점 있는 숫자. / 의 결과는 늘 이쪽

값은 같아도 타입이 달라요. 2파트에서 본 int와 float예요.

3. 사칙연산

몫 //, 나머지 %, 거듭제곱 **

사탕 7개를 3개씩 나누면



몫 2 (묶음 수)　　**나머지 1**

```
[3]  print(7 // 3)
     print(7 % 3)
     print(2 ** 3)
```

Python

```
2
1
8
```

//　몫

7 // 3　→　2

나누고 소수점 아래는 버려요

%　나머지

7 % 3　→　1

나누고 남은 수

****　거듭제곱**

2 ** 3　→　8

2 × 2 × 2

% 는 퍼센트가 아니라 나머지예요.

3. 사칙연산

곱하기를 더하기보다 먼저 계산해요

[4]
```python
2 + 3 * 4
```
Python

```
14
```

[5]
```python
(2 + 3) * 4
```
Python

```
20
```

`2 + 3 * 4` 3 * 4 먼저

`2 + 12` 그다음 더하기

`14`

`(2 + 3) * 4` 괄호 먼저

`5 * 4` 그다음 곱하기

`20`

먼저 하는 순서 `( )` → `**` → `* / // %` → `+ -`

헷갈리면 먼저 할 계산을 괄호로 묶어요.

3. 사칙연산

숫자에 이름을 붙여 계산해요

장보기 합계

[6]
```python
price = 3000
count = 2
total = price * count
print("합계:", total)
```

```
합계: 6000
```

세 사람이 나눠 내기

[7]
```python
total = 10000
people = 3
print("한 사람:", total // people)
print("남는 돈:", total % people)
```

```
한 사람: 3333
남는 돈: 1
```

`3000 * 2` 무슨 숫자인지 몰라요

`price * count` 가격 곱하기 개수

price만 바꾸고 다시 실행하면 합계도 새로 계산돼요.

3. 사칙연산

+= 는 지금 값에 더해서 다시 넣어요

[8]
```
score = 0
score += 10
score += 5
score
```
Python

```
15
```



`score += 10` 는 `score = score + 10` 를 줄인 거예요

같은 모양 `-=` `*=` `/=` 도 있어요

점수나 합계처럼 값을 쌓아 갈 때 써요.

3. 사칙연산

글자에도 + 와 * 를 써요

이어 붙여요

```
"kim" + "lee"
```

→ `'kimlee'`

3번 반복해요

```
"ha" * 3
```

→ `'hahaha'`

따옴표가 있으면 숫자도 글자예요

```
"3" + "3"
```

→ `'33'`

글자와 숫자는 더할 수 없어요

```
"3" + 3
```

→ `TypeError: can only concatenate str (not "int") to str`

글자와 숫자를 섞어 더하면 TypeError예요. int("3") + 3처럼 타입을 맞춰요.

4. 비교와 논리연산

비교하면 답은 True 아니면 False예요

[1]
```python
age = 25
print(age > 20)
print(age < 20)
print(age == 25)
print(age != 25)
print(age >= 25)
print(age <= 24)
```

```
True
False
True
False
True
False
```

>	크다	age > 20	True
<	작다	age < 20	False
==	같다	age == 25	True
!=	다르다	age != 25	False
>=	크거나 같다	age >= 25	True
<=	작거나 같다	age <= 24	False

결과는 10에서 본 불리언(bool)이에요. 같다는 = 를 두 번 써요.

4. 비교와 논리연산

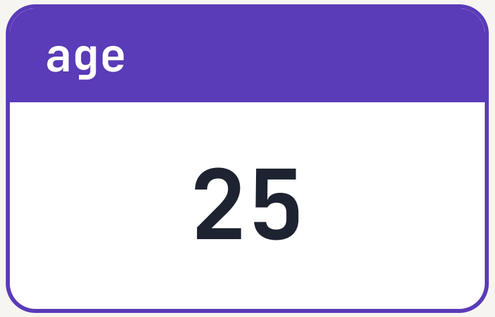
= 하나는 넣기, == 두 개는 묻기

== 묻기

[2]
```python
age == 30
```

```
False
```

age가 30이냐고 물어요.
age는 25 그대로예요.

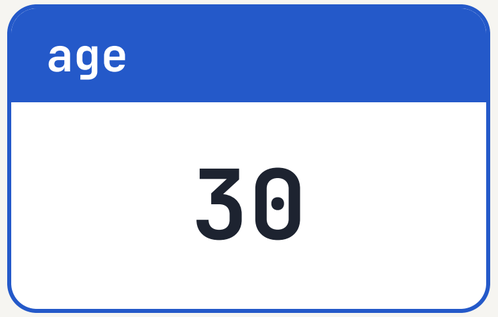


= 넣기

[3]
```python
age = 30
age
```

```
30
```

age에 30을 넣어요. 값이 바뀌어요.



비교하려고 = 를 하나만 쓰면 값이 바뀌어 버려요.

4. 비교와 논리연산

비교한 결과도 변수에 넣을 수 있어요

[4]
```python
age = 25
is_adult = age >= 20
print(is_adult)
print(type(is_adult))
```

```
True
<class 'bool'>
```

이름은 is_ 로 시작해요

10의 is_alive처럼 True, False를 담는
변수는 is_adult로 지으면 읽기 쉬워요.

`age >= 20`

25 >= 20 을 먼저 계산해요

↓ `True`

is_adult

True

= 오른쪽을 먼저 계산한다는 규칙은 비교에서도 같아요.

and 는 둘 다 True일 때만 True예요

[5]
```
print(True and True)
print(True and False)
print(False and True)
print(False and False)
```
Python

```
True
False
False
False
```

왼쪽		오른쪽		결과
True	and	True	→	True
True	and	False	→	False
False	and	True	→	False
False	and	False	→	False

하나라도 False면 결과는 False예요.

4. 비교와 논리연산

or 는 하나만 True여도 True예요

[6]
```python
print(True or True)
print(True or False)
print(False or True)
print(False or False)
```

```
True
True
True
False
```

왼쪽		오른쪽		결과
True	or	True	→	True
True	or	False	→	True
False	or	True	→	True
False	or	False	→	False

둘 다 False일 때만 False예요.

4. 비교와 논리연산

not 은 True와 False를 뒤집어요

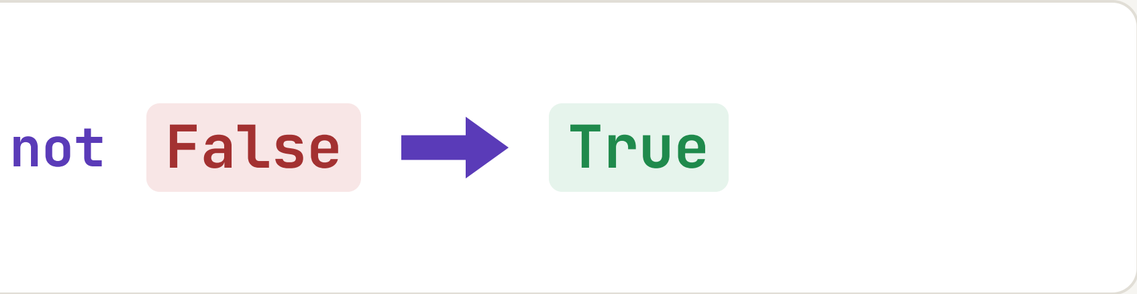
[7]
```python
print(not True)
print(not False)
```

```
False
True
```

앞에 한 번만 붙여요

and, or는 값 두 개 사이에 두고,
not은 값 하나 앞에 붙여요.





not 은 '아니다'예요. 바로 뒤의 값을 반대로 바꿔요.

4. 비교와 논리연산

놀이기구 탑승 조건을 and로 적어요

조건 · 키 120cm 이상 **그리고** 7살 이상

[8]
```python
height = 130
age = 9
height >= 120 and age >= 7
```

```
True
```

130 >= 120 → True

9 >= 7 → True

True and True → True

탈 수 있어요

[9]
```python
height = 110
height >= 120 and age >= 7
```

```
False
```

110 >= 120 → False

9 >= 7 → True

False and True → False

키가 모자라서 못 타요

조건을 모두 만족해야 하면 and예요.

4. 비교와 논리연산

할인 조건을 or로 적어요

조건 · 회원이거나 **또는** 5만 원 이상 샀으면 할인

[10]
```python
is_member = False
total = 60000
is_member or total >= 50000
```

```
True
```

`is_member` → `False`

`60000 >= 50000` → `True`

`False or True` → `True`

회원이 아니어도 할인돼요

[11]
```python
not is_member
```

```
True
```

`not is_member`

`not False` → `True`

회원이 아닌가요?

not을 붙이면 뜻도 반대로 읽어요.

둘 중 하나만 만족해도 되면 or예요.

4. 비교와 논리연산

계산, 비교, not, and, or 순서로 풀어요

[14]
```python
2 + 3 > 4 and not False
```

```
True
```

시작 `2 + 3 > 4 and not False`

1 계산 `5 > 4 and not False`

2 비교 `True and not False`

3 not `True and True`

4 and `True`

먼저 하는 순서

1. `+ - * /` 계산
2. `> == !=` 비교
3. `not` 뒤집기
4. `and` 그리고
5. `or` 또는

헷갈리면 (age >= 20) and (age < 30)처럼 괄호로 묶어요.

정리

막히면 오류의 마지막 줄을 봐요

1	`NameError: name 'Age' is not defined` 대문자와 소문자를 다르게 적었거나 넣는 셀을 안 돌렸어요	철자 확인, Run All
2	`NameError: name 'true' is not defined` True, False는 첫 글자가 대문자예요	True False
3	`TypeError: can only concatenate str (not "int") to str` 따옴표 글자와 숫자를 더했어요	int("3") + 3
4	`ValueError: invalid literal for int() with base 10: '삼'` int()가 숫자로 못 읽는 글자예요	소수 글자는 float()
5	`ZeroDivisionError: division by zero` 0으로 나눴어요	나누는 수 확인
6	**비교했는데 값이 바뀌었어요** = 를 하나만 썼어요	== 두 개
7	**6 / 2가 3.0으로 나와요** / 의 결과는 항상 실수예요	몫만 필요하면 //

오류가 나면 위쪽 Traceback보다 맨 아래 줄부터 읽어요.

한 장 요약

`age = 25`

오른쪽 값을 왼쪽 이름에 넣기

`print("나이:", age)`

설명 붙여 값 보기

`type(age)`

타입 확인

`int("3")  str(3)`

타입 바꾸기

`+ - * /`

사칙연산. / 는 항상 실수

`// % **`

몫, 나머지, 거듭제곱

`score += 10`

지금 값에 더해 다시 넣기

`== != > <`

비교하면 True 아니면 False

`and` 둘 다 True · `or` 하나라도 True · `not` 뒤집기

푸는 순서는 계산, 비교, not, and, or. 헷갈리면 괄호로 묶어요